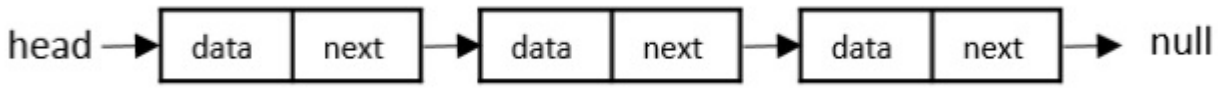
Linked List Data Structure

What is Linked List?

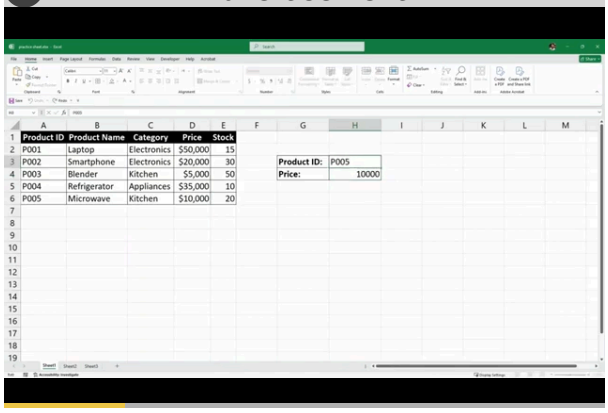
A linked list is a linear data structure which can store a collection of "nodes" connected together via links i.e. pointers. Linked lists nodes are not stored at a contiguous location, rather they are linked using pointers to the different memory locations. A node consists of the data value and a pointer to the address of the next node within the linked list.

A linked list is a dynamic linear data structure whose memory size can be allocated or de-allocated at run time based on the operation insertion or deletion, this helps in using system memory efficiently. Linked lists can be used to implment various data structures like a stack, queue, graph, hash maps, etc.



A linked list starts with a **head** node which points to the first node. Every node consists of data which holds the actual data (value) associated with the node and a next pointer which holds the memory address of the next node in the linked list. The last node is called the tail node in the list which points to **null** indicating the end of the list.

Advertisement



Linked Lists vs Arrays

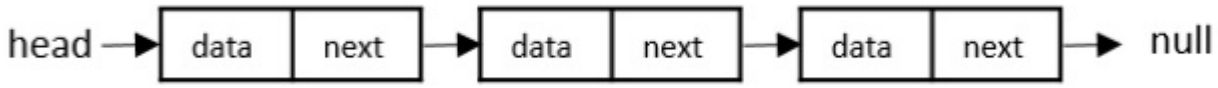
In case of arrays, the size is given at the time of creation and so arrays are of fixed lenghth where as Linked lists are dynamic in size and any number of nodes can be added in the linked lists dynamically. An array can accommodate similar types of data types where as linked lists can store various nodes of different data types.

Types of Linked List

Following are the various types of linked list.

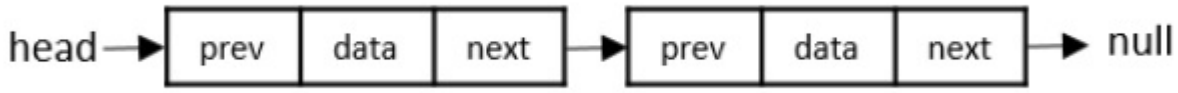
Singly Linked Lists

Singly linked lists contain two "buckets" in one node; one bucket holds the data and the other bucket holds the address of the next node of the list. Traversals can be done in one direction only as there is only a single link between two nodes of the same list.



Doubly Linked Lists

Doubly Linked Lists contain three "buckets" in one node; one bucket holds the data and the other buckets hold the addresses of the previous and next nodes in the list. The list is traversed twice as the nodes in the list are connected to each other from both sides.

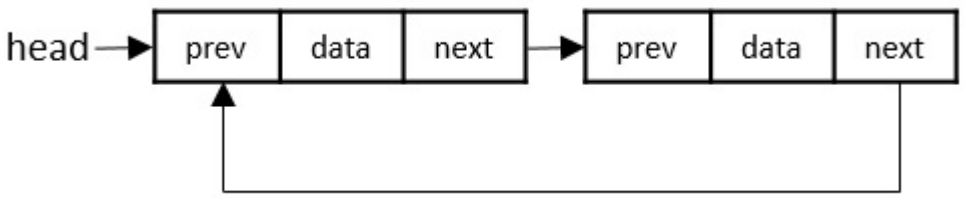


Advertisement

Circular Linked Lists

Circular linked lists can exist in both singly linked list and doubly linked list.

Since the last node and the first node of the circular linked list are connected, the traversal in this linked list will go on forever until it is broken.



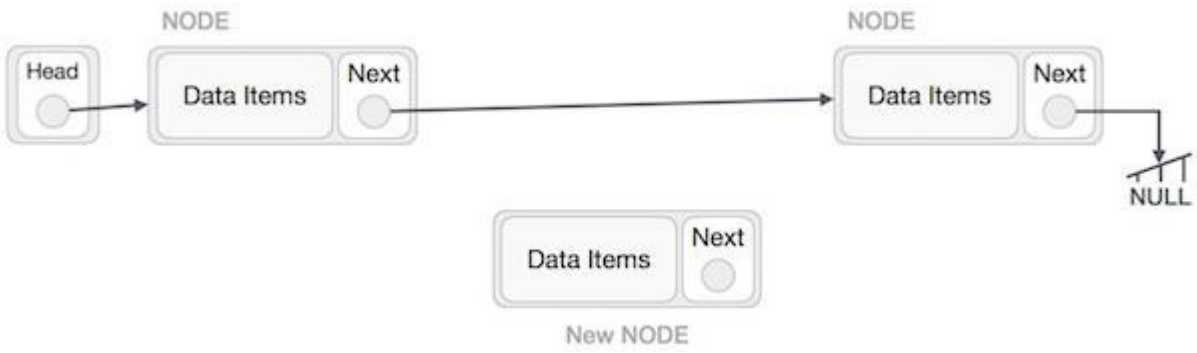
Basic Operations in Linked List

The basic operations in the linked lists are insertion, deletion, searching, display, and deleting an element at a given key. These operations are performed on Singly Linked Lists as given below –

- **Insertion** – Adds an element at the beginning of the list.
- **Deletion** – Deletes an element at the beginning of the list.
- **Display** – Displays the complete list.
- **Search** – Searches an element using the given key.
- **Delete** – Deletes an element using the given key.

Linked List - Insertion Operation

Adding a new node in linked list is a more than one step activity. We shall learn this with diagrams here. First, create a node using the same structure and find the location where it has to be inserted.

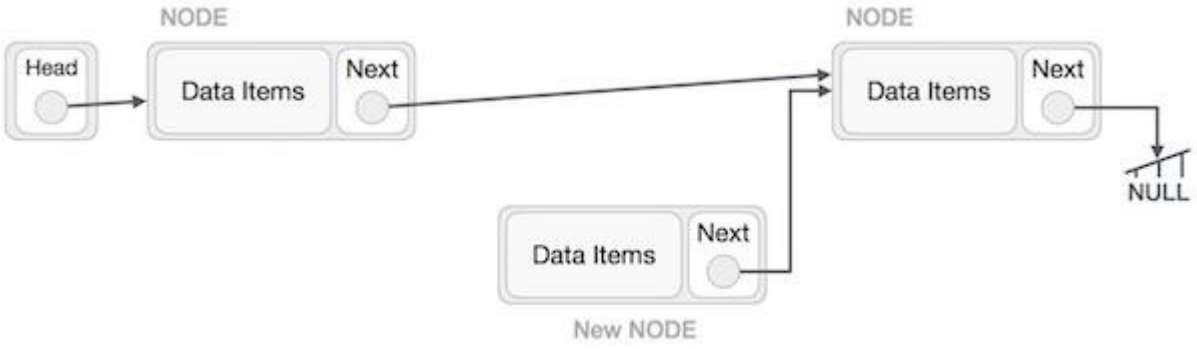


Im[illegible]ewNode), between A (LeftNode) and C (RightNode). Then point B.next to C –

Advertisement

```
NewNode.next -> RightNode;
```

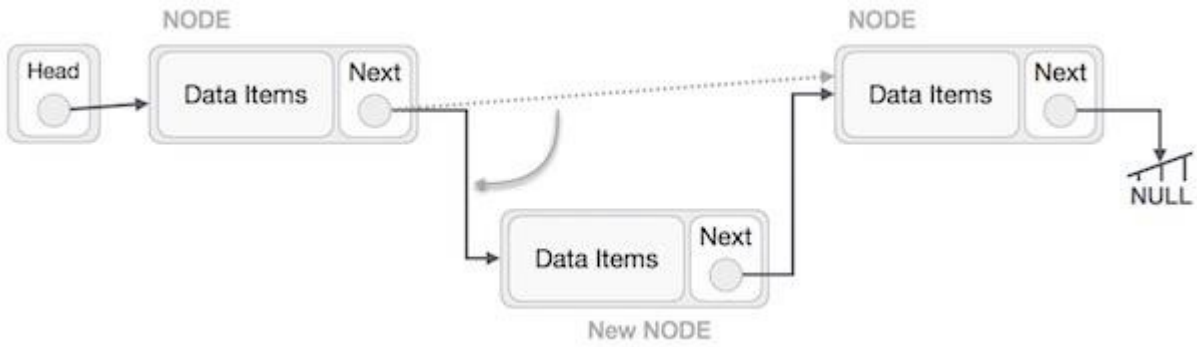
It should look like this –



Now, the next node at the left should point to the new node.

```
LeftNode.next -> NewNode;
```

This will put the new node in the middle of the two. The new list should look like this –



Insertion in linked list can be done in three different ways. They are explained as follows –

Insertion at Beginning

In this operation, we are adding an element at the beginning of the list.

Algorithm

```
1. START
2. Create a node to store the data
3. Check if the list is empty
4. If the list is empty, add the data to the node and
   assign the head pointer to it.
5. If the list is not empty, add the data to a node and link to the
   current head. Assign the head to the newly added node.
6. END
```

Advertisement

Example

Following are the implementations of this operation in various programming languages –

C C++ Java Python

```python
class Node:
   def __init__(self, data=None):
      self.data = data
      self.next = None
class SLL:
   def __init__(self):
      self.head = None

# Print the linked list
   def listprint(self):
      printval = self.head
      print("Linked List: ")
      while printval is not None:
         print (printval.data)
         printval = printval.next
   def AddAtBeginning(self,newdata):
      NewNode = Node(newdata)

      # Update the new nodes next val to existing node
      NewNode.next = self.head
      self.head = NewNode

l1 = SLL()
l1.head = Node("731")
e2 = Node("672")
e3 = Node("63")

l1.head.next = e2
e2.next = e3
```

Advertisement

```python
l1.AddAtBeginning("122")
l1.listprint()
```

Output

```
Linked List:
122
731
```

```
672
63
```

Insertion at Ending

In this operation, we are adding an element at the ending of the list.

Algorithm

```
1. START
2. Create a new node and assign the data
3. Find the last node
4. Point the last node to new node
5. END
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | Python

```python
class Node:
   def __init__(self, data=None):
      self.data = data
      self.next = None
class LL:
   def __init__(self):
      self.head = None
   def listprint(self):
      val = self.head
      print("Linked List:")
      while val is not None:
         print(val.data)
         val = val.next

l1 = LL()
l1.head = Node("23")
l2 = Node("12")
l3 = Node("7")
l4 = Node("14")
l5 = Node("61")
```

Advertisement

```
# Linking the first Node to second node
l1.head.next = l2

# Linking the second Node to third node
l2.next = l3
l3.next = l4
l4.next = l5
l1.listprint()
```

Output

```
Linked List:
23
12
7
14
61
```

Insertion at a Given Position

In this operation, we are adding an element at any position within the list.

Algorithm

```
1. START
2. Create a new node and assign data to it
3. Iterate until the node at position is found
4. Point first to new first node
5. END
```

Example

Following are the implementations of this operation in various programming languages –

Advertisement

C C++ Java Python

```
class Node:
   def __init__(self, data=None):
      self.data = data
      self.next = None
```

```python
class SLL:
   def __init__(self):
      self.head = None

# Print the linked list
   def listprint(self):
      printval = self.head
      print("Linked List: ")
      while printval is not None:
         print (printval.data)
         printval = printval.next

   # Function to add node
   def InsertAtPos(self,nodeatpos,newdata):
      if nodeatpos is None:
         print("The mentioned node is absent")
         return
      NewNode = Node(newdata)
      NewNode.next = nodeatpos.next
      nodeatpos.next = NewNode

l1 = SLL()
l1.head = Node("731")
e2 = Node("672")
e3 = Node("63")

l1.head.next = e2
e2.next = e3

l1.InsertAtPos(l1.head.next, "122")
l1.listprint()
```

O

Advertisement

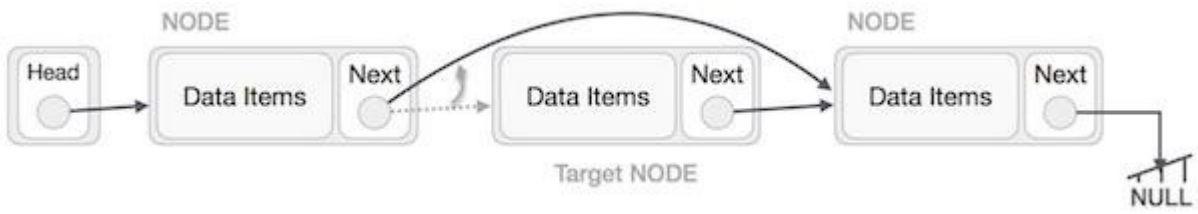
```
Linked List:
731
672
122
63
```

Linked List - Deletion Operation

Deletion is also a more than one step process. We shall learn with pictorial representation. First, locate the target node to be removed, by using searching algorithms.

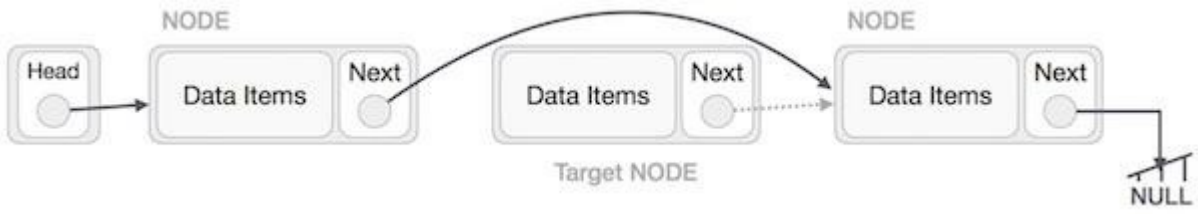
The left (previous) node of the target node now should point to the next node of the target node –

```
LeftNode.next -> TargetNode.next;
```



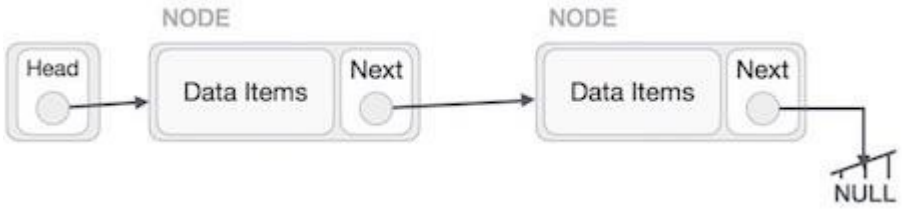
This will remove the link that was pointing to the target node. Now, using the following code, we will remove what the target node is pointing at.

```
TargetNode.next -> NULL;
```



We need to use the deleted node. We can keep that in memory otherwise we can simply deallocate memory and wipe off the target node completely.

Advertisement



Similar steps should be taken if the node is being inserted at the beginning of the list. While inserting it at the end, the second last node of the list should point to the new node and the new node will point to NULL.

Deletion in linked lists is also performed in three different ways. They are as follows –

Deletion at Beginning

In this deletion operation of the linked, we are deleting an element from the beginning of the list. For this, we point the head to the second node.

Algorithm

```
1. START
2. Assign the head pointer to the next node in the list
3. END
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | Python

```python
#python code for deletion at beginning using linked list.
from typing import Optional
class Node:
    def __init__(self, data: int, next: Optional['Node'] = None):
        self.data = data
        self.next = next
```

Advertisement

```python
    def __init__(self):
        self.head = None
     #display the list
    def print_list(self):
        p = self.head
        print("\n[", end="")
        while p:
            print(f" {p.data} ", end="")
```

```
            p = p.next
        print("]")
     #Insertion at the beginning
    def insert_at_begin(self, data: int):
        lk = Node(data)
         #point it to old first node
        lk.next = self.head
        #point firt to new first node
        self.head = lk
    def delete_at_begin(self):
        self.head = self.head.next
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert_at_begin(12)
    linked_list.insert_at_begin(22)
    linked_list.insert_at_begin(30)
    linked_list.insert_at_begin(44)
    linked_list.insert_at_begin(50)
    #print list
    print("Linked List: ", end="")
    linked_list.print_list()
    linked_list.delete_at_begin()
    print("Linked List after deletion: ", end="")
    linked_list.print_list()
```

Output

```
Linked List:
[ 50  44  30  22  12 ]
Linked List after deletion:
[ 44  30  22  12 ]
```

Deletion at Ending

Advertisement

In this deletion operation of the linked, we are deleting an element from the ending of the list.

Algorithm

```
1. START
2. Iterate until you find the second last element in the list.
```

```
3. Assign NULL to the second last element in the list.
4. END
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | Python

```python
#python code for deletion at beginning using linked list.
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
 #Displaying the list
    def printList(self):
        p = self.head
        print("\n[", end="")
        while p != None:
            print(" " + str(p.data) + " ", end="")
            p = p.next
        print("]")
 #Insertion at the beginning
    def insertatbegin(self, data):
        #create a link
        lk = Node(data)
        #point it to old first node
        lk.next = self.head
        #point first to new first node
        self.head = lk
```

Advertisement

```python
    def deleteatend(self):
        linkedlist = self.head
        while linkedlist.next.next != None:
            linkedlist = linkedlist.next
        linkedlist.next = None
if __name__ == "__main__":
    linked_list = LinkedList()
```

```
    linked_list.insertatbegin(12)
    linked_list.insertatbegin(22)
    linked_list.insertatbegin(30)
    linked_list.insertatbegin(40)
    linked_list.insertatbegin(55)
    #print list
    print("Linked List: ", end="")
    linked_list.printList()
    linked_list.deleteatend()
    print("Linked List after deletion: ", end="")
    linked_list.printList()
```

Output

```
Linked List:
[ 55  40  30  22  12 ]
Linked List after deletion:
[ 55  40  30  22 ]
```

Deletion at a Given Position

In this deletion operation of the linked, we are deleting an element at any position of the list.

Algorithm

```
1. START
2. Iterate until find the current node at position in the list.
3. Assign the adjacent node of current node in the list
   to its previous node.
4. END
```

Example

Following are the implementations of this operation in various programming languages –

Advertisement

C | C++ | Java | Python

```
#python code for deletion at given position using linked list.
class Node:
    def __init__(self, data=None):
        self.data = data
```

```
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    # display the list
    def printList(self):
        p = self.head
        print("\n[", end="")
        #start from the beginning
        while(p != None):
            print(" ", p.data, " ", end="")
            p = p.next
        print("]")
    #insertion at the beginning
    def insertatbegin(self, data):
        #create a link
        lk = Node(data)
        # point it to old first node
        lk.next = self.head
        #point first to new first node
        self.head = lk
    def deletenode(self, key):
        temp = self.head
        if (temp != None and temp.data == key):
            self.head = temp.next
            return
        # Find the key to be deleted
        while (temp != None and temp.data != key):
            prev = temp
            temp = temp.next
        # If the key is not present
        if (temp == None):
            return
        # Remove the node
```

Advertisement

```
        prev.next = temp.next
llist = LinkedList()
llist.insertatbegin(12)
llist.insertatbegin(22)
llist.insertatbegin(30)
llist.insertatbegin(40)
llist.insertatbegin(55)
print("Original Linked List: ", end="")
```

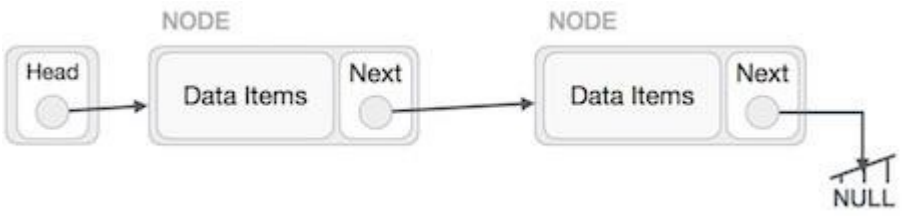
```
# print list
llist.printList()
llist.deletenode(30)
print("Linked List after deletion: ", end="")
# print list
llist.printList()
```

Output

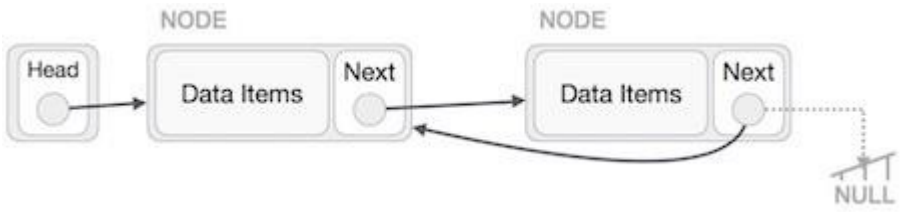
```
Original Linked List:
[  55    40    30    22    12  ]
Linked List after deletion:
[  55    40    22    12  ]
```

Linked List - Reversal Operation

This operation is a thorough one. We need to make the last node to be pointed by the head node and reverse the whole linked list.



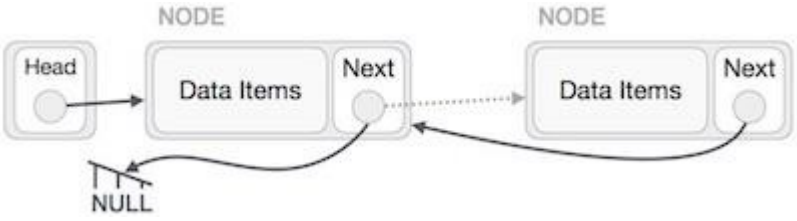
First, we traverse to the end of the list. It should be pointing to NULL. Now, we shall make it point to its previous node –



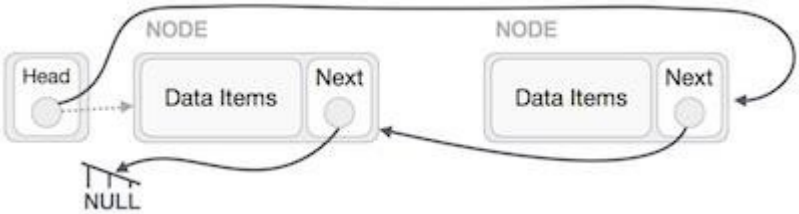
We have to make sure that the last node is not the last node. So we'll have some temp node, which looks like the head node pointing to the last node. Now, we shall make all lef

Advertisement

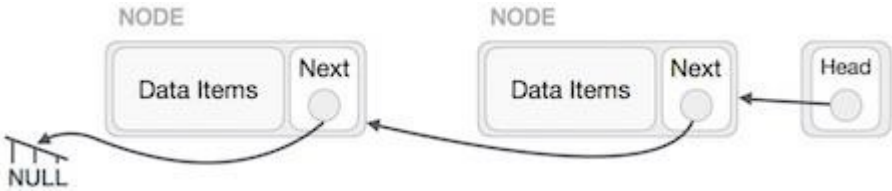
es one by one.



Except the node (first node) pointed by the head node, all nodes should point to their predecessor, making them their new successor. The first node will point to NULL.



We'll make the head node point to the new first node by using the temp node.



Algorithm

Step by step process to reverse a linked list is as follows –

```
1. START
2. We use three pointers to perform the reversing:
   prev, next, head.
3. Point the current node to head and assign its next value to
   the prev node.
4. Iteratively repeat the step 3 for all the nodes in the list.
5. Assign head to the prev node.
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | Python

```
class Node:
```

Advertisement

```
      self.data = data
      self.next = None

class SLL:
   def __init__(self):
      self.head = None

# Print the linked list
```

```
    def listprint(self):
        printval = self.head
        print("Linked List: ")
        while printval is not None:
            print (printval.data)
            printval = printval.next
    def reverse(self):
        prev = None
        curr = self.head
        while(curr is not None):
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next
        self.head = prev

l1 = SLL()
l1.head = Node("731")
e2 = Node("672")
e3 = Node("63")

l1.head.next = e2
e2.next = e3

l1.listprint()
l1.reverse()
print("After reversing: ")
l1.listprint()
```

Output

```
Linked List:
731
672
63
After reversing:
Linked List:
63
672
731
```

Advertisement

Linked List - Search Operation

Searching for an element in the list using a key element. This operation is done in the same way as array search; comparing every element in the list with the key element given.


```
START
If the list is not empty, iteratively check if the list contains the key
If the key element is not present in the list, unsuccessful search
END
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | Python

```python
class Node:
   def __init__(self, data=None):
      self.data = data
      self.next = None

class SLL:
   def listprint(self):
      printval = self.head
      print("Linked List: ")
      while printval is not None:
         print (printval.data)
         printval = printval.next
   def __init__(self):
      self.head = None
   def search(self, x):
      count = 0

      # Initialize current to head
```

Advertisement

```
        current = self.head

        # loop till current not equal to None
        while current != None:
            if current.data == x:
                print("Element is found")
                count = count + 1
            current = current.next
        if count == 0:
            print("Element is not found in the list")

l1 = SLL()
l1.head = Node("731")
e2 = Node("672")
e3 = Node("63")

l1.head.next = e2
e2.next = e3
l1.listprint()
ele = "63"
print("Element to be searched is: ", ele);
l1.search(ele)
```

Output

```
Linked List:
731
672
63
Element to be searched is:  63
Element is found
```

Linked List - Traversal Operation

Advertisement

The traversal operation walks through all the elements of the list in an order and displays the elements in that order.

Algorithm

```
1. START
2. While the list is not empty and did not reach the end of the list,
```

```
   print the data in each node
3. END
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | Python

```python
class Node:
   def __init__(self, data=None):
      self.data = data
      self.next = None
class SLL:
   def __init__(self):
      self.head = None

# Print the linked list
   def listprint(self):
      printval = self.head
      print("Linked List: ")
      while printval is not None:
         print (printval.data)
         printval = printval.next

l1 = SLL()
l1.head = Node("731")
e2 = Node("672")
e3 = Node("63")

l1.head.next = e2
```

Advertisement

```python
l1.listprint()
```

Output

```
Linked List:
731
```

```
672
63
```

Linked List - Complete implementation

Following are the complete implementations of Linked List in various programming languages –

C | C++ | Java | Python

```python
class LLNode:
   def __init__(self, data=None):
      self.data = data
      self.next = None
class LL:
   def __init__(self):
      self.head = None
   def listprint(self):
      printval = self.head
      while printval is not None:
         print(printval.data)
         printval = printval.next
   def AddAtBeginning(self,newdata):
      NewNode = LLNode(newdata)

      # Update the new nodes next val to existing node
      NewNode.next = self.head
      self.head = NewNode

   # Function to add node at a position
   def InsertAtPos(self,nodeatpos,newdata):
```

Advertisement

```python
         print("The mentioned node is absent")
         return
      NewNode = LLNode(newdata)
      NewNode.next = nodeatpos.next
      nodeatpos.next = NewNode
   def reverse(self):
      prev = None
      curr = self.head
```

```
        while(curr is not None):
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next
        self.head = prev
    def search(self, x):
        count = 0

        # Initialize current to head
        current = self.head

        # loop till current not equal to None
        while current != None:
            if current.data == x:
                print("Element is found")
                count = count + 1
            current = current.next
        if count == 0:
            print("Element is not found in the list")

l1 = LL()
l1.head = LLNode("23")
l2 = LLNode("12")
l3 = LLNode("7")
l4 = LLNode("14")
l5 = LLNode("61")

# Linking the first Node to second node
l1.head.next = l2

# Linking the second Node to third node
l2.next = l3
l3.next = l4
l4.next = l5
print("Original Linked List: ")
l1.listprint()

l1.AddAtBeginning("45")
l1.InsertAtPos(l1.head.next.next, "4")
print("Updated Linked List:")
l1.listprint()
```

Advertisement

```
l1.reverse()

print("Reversed Linked List:")
l1.listprint()
l1.search("7")
```

Output

```
Original Linked List:
23
12
7
14
61
Updated Linked List:
45
23
12
4
7
14
61
Reversed Linked List:
61
14
7
4
12
23
45
Element is found
```

Advertisement

TOP TUTORIALS

Python Tutorial

Java Tutorial

C++ Tutorial

C Programming Tutorial

C# Tutorial

PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Advertisement
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler

Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

ABOUT US | OUR TEAM | CAREERS | JOBS | CONTACT US | TERMS OF USE |
PRIVACY POLICY | REFUND POLICY | COOKIES POLICY | FAQ'S

tutorialspoint

GET IT ON Google Play
Download on the App Store

Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2025. All Rights Reserved.

Advertisement